[RAG] Smart Document Analyzer

Retrieval-Augmented Generation (RAG) System

Project Report · Version 1.0.0 · June 2026

Stack: FastAPI (Python) + Next.js (TypeScript)

FastAPI	ChromaDB	LangChain	HuggingFace	Ollama	Next.js	SQLite	JWT

Smart Document Analyzer — Project Report

> ***Version:*** *1.0.0 | * ***Date:*** *June 2026 | * ***Stack:*** *Python (FastAPI) + Next.js (TypeScript)*

1. Project Overview

Smart Document Analyzer is a full-stack, AI-powered Retrieval-Augmented Generation (RAG) application that allows users to upload their own documents (PDF, TXT) and interactively ask natural-language questions about their content. The system retrieves relevant passages from the documents and generates accurate, context-grounded answers using a locally-running Large Language Model (LLM), avoiding hallucinations by strictly constraining the model to only what the documents contain.

1.1 Motivation

Traditional search systems return raw document snippets that require users to manually read and synthesize information. This project bridges that gap by combining:

- **Semantic search** (vector embeddings) to find the *most relevant* passages
- **LLM generation** to formulate a *natural-language answer* from those passages
- **Per-user isolation** so that each user's documents are private and inaccessible to others

2. Objectives

#	Objective
1	Enable users to securely upload and manage documents (PDF, TXT)
2	Automatically extract, chunk, and embed document content into a vector database
3	Allow users to query their documents in natural language via a chat interface
4	Stream LLM-generated answers in real-time for a responsive UX
5	Display source passages (citations) alongside each answer
6	Isolate user data — each user can only query their own documents
7	Support optional document-level filtering in queries

3. System Architecture

```
flowchart TD
subgraph Frontend["Frontend (Next.js · TypeScript)"]
UI_Login["Login / Register Page"]
UI_Dashboard["Dashboard — Document Manager"]
UI_Chat["Chat Page — RAG Q&A"]
end

subgraph Backend["Backend (FastAPI · Python)"]
API_Auth["/api/v1/auth"]
API_Docs["/api/v1/documents"]
API_Chat["/api/v1/chat"]
DocProc["Document Processor Service"]
end

subgraph Storage["Storage Layer"]
SQLite["SQLite Database\n(Users · Documents)"]
ChromaDB["ChromaDB\n(Vector Store)"]
FS["File System\nuploaded_docs/"]
end

subgraph AI["AI Layer"]
Embedder["HuggingFace Embeddings\n(all-MiniLM-L6-v2)"]
Ollama["Ollama LLM\n(llama3.2:1b — local)"]
end

UI_Login -->|OAuth2 Login / Register| API_Auth
UI_Dashboard -->|Upload / List / Delete| API_Docs
UI_Chat -->|Streaming Chat POST| API_Chat

API_Auth --> SQLite
API_Docs --> SQLite
API_Docs --> FS
API_Docs --> DocProc

DocProc --> Embedder
DocProc --> ChromaDB

API_Chat --> ChromaDB
API_Chat --> Ollama
```

---

4. Technology Stack

4.1 Backend

Layer	Technology	Purpose
Web Framework	FastAPI 0.x	REST API server with async support

Layer	Technology	Purpose
ORM / DB	SQLAlchemy + SQLite	User & document metadata persistence
Authentication	JWT (python-jose) + bcrypt	Secure stateless token auth
Document Loading	LangChain (PyPDFLoader, TextLoader)	Extract text from PDF and TXT files
Text Chunking	LangChain RecursiveCharacterTextSplitter	Split documents into overlapping chunks
Embeddings	HuggingFace `all-MiniLM-L6-v2`	Generate dense vector representations
Vector Store	ChromaDB (persisted)	Store and retrieve document embeddings
LLM	Ollama (`llama3.2:1b`) running locally	Generate contextual answers
LLM Orchestration	LangChain Core (PromptTemplate, chains)	Build the RAG generation pipeline
Streaming	FastAPI StreamingResponse + SSE	Real-time token-by-token answer delivery
Validation	Pydantic v2 (pydantic-settings)	Request/response schemas and config
CORS	FastAPI CORSMiddleware	Enable cross-origin requests from frontend

4.2 Frontend

Layer	Technology	Purpose
Framework	Next.js 14 (App Router)	SSR/CSR hybrid React framework
Language	TypeScript	Type-safe frontend development
State Management	Zustand (`authStore`)	Global authentication state
HTTP Client	Axios with interceptors	Auto-attach JWT tokens; handle 401s
File Upload	react-dropzone	Drag & drop file upload UI
Animations	Framer Motion	Page transitions, message animations
Icons	Lucide React	Consistent icon set
Streaming	Native Fetch API + ReadableStream	Consume SSE stream from chat endpoint
Styling	Tailwind CSS	Utility-first styling with dark theme

5. Module Descriptions

5.1 Backend Modules

`app/main.py` — ***Application Entry Point***

Initialises the FastAPI app, registers CORS middleware, creates all SQLAlchemy database tables on startup, and mounts the three main routers under the `/api/v1` prefix.

`app/core/config.py` — ***Configuration***

Uses `pydantic-settings` to load all configuration from environment variables (`.env` file). Key settings:

Setting	Default Value	Description
`PROJECT_NAME`	Smart Document Analyzer API	App display name
`VERSION`	1.0.0	API version
`API_V1_STR`	/api/v1	URL prefix
`SECRET_KEY`	(change in production!)	JWT signing key
`ACCESS_TOKEN_EXPIRE_MINUTES`	11,520 min (8 days)	Token TTL
`DATABASE_URL`	sqlite:///./sql_app.db	SQLite database path
`CHROMA_PERSIST_DIR`	./chroma_db	ChromaDB storage directory
`OLLAMA_BASE_URL`	http://localhost:11434	Ollama server URL
`OLLAMA_MODEL`	llama3.2:1b	LLM model name

`app/core/security.py` — ***Authentication & Security***

- `get_password_hash(password)` — bcrypt hashes a plaintext password
- `verify_password(plain, hashed)` — bcrypt verification
- `create_access_token(subject, expires_delta)` — creates a signed HS256 JWT
- `get_current_user(token, db)` — FastAPI dependency that decodes JWT and fetches the user from the database; raises HTTP 403 on invalid token

---

`app/api/auth.py` — ***Authentication Endpoints***

Method	Path	Description
`POST`	`/api/v1/auth/login`	OAuth2 form login → returns JWT bearer token
`POST`	`/api/v1/auth/register`	Register new user (email + username + password)

`app/api/documents.py` — *Document Management Endpoints*

Method	Path	Description
`POST`	`/api/v1/documents/upload`	Upload a PDF/TXT file; save to disk; trigger RAG processing
`GET`	`/api/v1/documents/`	List all documents belonging to the current user
`DELETE`	`/api/v1/documents/{id}`	Delete document from filesystem, ChromaDB, and SQLite

Each uploaded file is stored with a **UUID-based filename** to prevent collisions and path-traversal attacks.

`app/api/chat.py` — *RAG Chat Endpoint*

Method	Path	Description
`POST`	`/api/v1/chat/`	Query documents; stream answer via Server-Sent Events

Flow: 1. Retrieves relevant document chunks from ChromaDB filtered by `user_id` (and optionally `document_id`s) 2. Formats chunks as context and passes them through a strict prompt template 3. Streams the LLM response token-by-token as SSE events:

- First event: `{"sources": [...]}` — cited passages
- Subsequent events: `{"chunk": "..."}` — answer tokens
- Final event: `[DONE]`

`app/services/document_processor.py` — *RAG Pipeline Service*

The core service implementing the 3-step RAG ingestion pipeline:

```
File (PDF/TXT)
■
▼
① Text Extraction ← LangChain Loaders (PyPDFLoader / TextLoader)
■
▼
② Chunking ← RecursiveCharacterTextSplitter
chunk_size=1000 (1000 chars with 200-char overlap)
chunk_overlap=200
■
▼
③ Embedding & Storage ← HuggingFace all-MiniLM-L6-v2 → ChromaDB
Metadata: document_id, user_id, chunk_index
```

Each chunk is tagged with `document_id`, `user_id`, and `chunk_index` metadata to enable **per-user filtering** and **per-document filtering** at query time.

5.2 Database Models

`User` ***Table (***`users`***)***

Column	Type	Constraints
`id`	Integer	PK, auto-increment
`username`	String	Unique, indexed, not null
`email`	String	Unique, indexed, not null
`hashed_password`	String	Not null
`created_at`	DateTime	Default: UTC now

Relationship: One-to-many with `Document` (cascade delete)

`Document` ***Table (***`documents`***)***

Column	Type	Constraints
`id`	Integer	PK, auto-increment
`file_name`	String	Indexed, not null (original filename)
`file_path`	String	Not null (UUID path on disk)
`upload_date`	DateTime	Default: UTC now
`user_id`	Integer	FK → `users.id`

5.3 Frontend Modules

Pages & Routes

Route	Component	Description
`/`	`page.tsx`	Root — redirects to `/login` or `/dashboard`
`/login`	`login/page.tsx`	Login form with JWT token storage
`/register`	`register/page.tsx`	User registration form
`/dashboard`	`dashboard/page.tsx`	Document upload & management
`/dashboard/chat`	`dashboard/chat/page.tsx`	RAG chat interface

`src/services/api.ts` ***— Axios HTTP Client***

A pre-configured Axios instance with two interceptors: 1. **Request interceptor** — reads `access_token` from `localStorage` and injects the `Authorization: Bearer <token>` header on every request 2. **Response interceptor** — on HTTP 401, clears the stored token and redirects to `/login`

`src/app/dashboard/layout.tsx` ***— Protected Layout***

Wraps all `/dashboard` routes. Checks `isAuthenticated` from the Zustand auth store; redirects unauthenticated users to `/login`. Renders the persistent **sidebar** with:

- App logo (SmartDoc)
- Navigation links (Documents, Chat) with Framer Motion animated active tab indicator
- User avatar (gradient initials) and Sign Out button

`src/app/dashboard/page.tsx` ***— Document Manager***

- **Drag-and-drop upload zone** (react-dropzone) accepting `.pdf` and `.txt`
- Animated upload status (spinner while processing)
- **Document grid** — responsive card list showing filename, upload date, and hover-revealed delete button with confirmation
- Framer Motion fade/scale animation on new document cards

`src/app/dashboard/chat/page.tsx` ***— Chat Interface***

- **Document selector bar** — pill-shaped toggle buttons to restrict queries to specific documents
- **Streaming message rendering** — consumes SSE stream from the backend and appends tokens to the assistant message in real-time
- **Source citations panel** — horizontally scrollable cards showing the retrieved source passages and their metadata
- Auto-scroll to the latest message
- Animated message bubbles (user: indigo, assistant: zinc)

---

6. RAG Pipeline — Detailed Flow

```
User submits a question (e.g. "What are the key findings?")
■
▼
1. RETRIEVAL (ChromaDB)
- Query embedded with all-MiniLM-L6-v2
- Cosine similarity search
- Filter: user_id = current user
- Optional filter: document_id IN [selected IDs]
- Returns top-k most relevant chunks
■
▼
2. CONTEXT ASSEMBLY
- Retrieved chunks concatenated as "Context"
- Injected into a strict prompt template:
"Use ONLY the provided context. Do not hallucinate."
■
▼
3. GENERATION (Ollama llama3.2:1b)
- LangChain async streaming chain
- Tokens streamed as SSE chunks to the browser
```

```
■
▼
4. RESPONSE TO USER
- Sources: cited passages with metadata
- Answer: streamed tokens rendered in real-time
```

---

7. API Reference

Authentication

```
POST /api/v1/auth/register
Body: { "username": "...", "email": "...", "password": "..." }
Response: User object

POST /api/v1/auth/login
Body: form-data { username (email), password }
Response: { "access_token": "...", "token_type": "bearer" }
```

Documents

```
POST /api/v1/documents/upload multipart/form-data; file=<file>
GET /api/v1/documents/ → [{ id, file_name, upload_date }]
DELETE /api/v1/documents/{document_id} → { message }
```

All document endpoints require `Authorization: Bearer <token>`.

Chat

```
POST /api/v1/chat/
Body: { "query": "...", "document_ids": [1, 2] } // document_ids optional
Response: text/event-stream (SSE)
data: {"sources": [...]}
data: {"chunk": "token..."}
data: [DONE]
```

---

8. Security Design

Concern	Implementation
Password storage	bcrypt with auto-generated salt (not MD5/SHA1)
Token-based auth	JWT signed with HS256; 8-day expiry
Data isolation	All DB and ChromaDB queries filtered by `user_id`
File naming	UUID filenames prevent path-traversal & enumeration

Concern	Implementation
Token refresh	401 responses trigger automatic logout + redirect
CORS	Enabled for all origins in development (restrict in production)

> **[WARNING]**
> The `SECRET_KEY` in `config.py` is a placeholder. **Must be replaced** with a securely generated random key before deploying to production.

---

9. Project File Structure

```
Smart Document Analyzer (RAG)/
■■■ backend/
■ ■■■ app/
■ ■ ■■■ main.py # FastAPI app entry point
■ ■ ■■■ api/
■ ■ ■ ■■■ auth.py # Login & Register endpoints
■ ■ ■ ■■■ documents.py # Upload, list, delete documents
■ ■ ■ ■■■ chat.py # RAG streaming chat endpoint
■ ■ ■■■ core/
■ ■ ■ ■■■ config.py # App settings (pydantic-settings)
■ ■ ■ ■■■ security.py # JWT, bcrypt, get_current_user
■ ■ ■■■ database/
■ ■ ■ ■■■ database.py # SQLAlchemy engine & session
■ ■ ■■■ models/
■ ■ ■ ■■■ user.py # User ORM model
■ ■ ■ ■■■ document.py # Document ORM model
■ ■ ■■■ schemas/
■ ■ ■ ■■■ user.py # UserCreate, User schemas
■ ■ ■ ■■■ token.py # Token, TokenPayload schemas
■ ■ ■ ■■■ chat.py # ChatRequest, ChatResponse schemas
■ ■ ■■■ services/
■ ■ ■■■ document_processor.py # RAG ingestion pipeline
■ ■■■ chroma_db/ # Persisted ChromaDB vector store
■ ■■■ uploaded_docs/ # Uploaded files (UUID-named)
■ ■■■ sql_app.db # SQLite database
■ ■■■ requirements.txt # Python dependencies
■
■■■ frontend/
■■■ src/
■ ■■■ app/
■ ■ ■■■ layout.tsx # Root layout
■ ■ ■■■ page.tsx # Root redirect
■ ■ ■■■ login/page.tsx # Login page
■ ■ ■■■ register/page.tsx # Registration page
■ ■ ■■■ dashboard/
■ ■ ■■■ layout.tsx # Protected sidebar layout
■ ■ ■■■ page.tsx # Document manager
■ ■ ■■■ chat/page.tsx # RAG chat interface
```

```
■ ■■■ services/
■ ■ ■■■ api.ts # Axios client with JWT interceptors
■ ■■■ store/
■ ■ ■■■ authStore.ts # Zustand authentication store
■ ■■■ utils/
■ ■■■ cn.ts # Tailwind class merging utility
■■■ package.json
■■■ next.config.ts
```

---

10. Setup & Deployment

Prerequisites

- Python 3.10+
- Node.js 18+
- [Ollama](https://ollama.com) installed and running locally

Backend Setup

```
# Navigate to backend
cd "Smart Document Analyzer (RAG)/backend"

# Create virtual environment
python -m venv venv
venv\Scripts\activate # Windows

# Install dependencies
pip install -r requirements.txt

# Pull the LLM model (requires Ollama)
ollama pull llama3.2:1b

# Start the FastAPI server
uvicorn app.main:app --reload --port 8000
```

API available at: `http://localhost:8000` Interactive docs: `http://localhost:8000/api/v1/openapi.json`

Frontend Setup

```
# Navigate to frontend
cd "Smart Document Analyzer (RAG)/frontend"

# Install dependencies
npm install

# Start the Next.js development server
npm run dev
```

Frontend available at: `http://localhost:3000`

---

11. Key Design Decisions

Decision	Rationale
Local LLM (Ollama)	Keeps all data on-premise; no API costs; full privacy
ChromaDB persisted	Vector store survives server restarts without re-indexing
Chunk overlap = 200	Prevents splitting of contextual sentences at chunk boundaries
UUID filenames	Prevents filename collisions and path-traversal vulnerabilities
SSE streaming	Delivers real-time LLM output without long polling or WebSockets
Per-user ChromaDB filter	Ensures strict data isolation at the retrieval layer
Lazy LLM initialization	Avoids loading the LLM model at startup unless actually needed

---

12. Known Limitations & Future Improvements

Area	Current State	Suggested Improvement
Background processing	Document processing is synchronous (blocks upload response)	Use Celery + Redis for async task queue
DOCX support	Code checks for `.docx` but processor only handles PDF/TXT	Add `python-docx` loader
Chunk size	Fixed 1000 chars for all documents	Allow per-document or per-user configuration
LLM model	Single `llama3.2:1b` model	Allow model selection per user
Production security	`SECRET_KEY` is hardcoded; CORS is `*`	Use env vars; restrict CORS origins
Error handling	Document processing errors are silently swallowed	Expose processing status in DB and UI
Token refresh	No refresh token — user must re-login after 8 days	Implement refresh token rotation
Search results	Top-k defaults	Allow users to configure retrieval depth

---

Report generated automatically from source code analysis · Smart Document Analyzer v1.0.0